HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Faculty of Computer Science and Engineering

CAPSTONE PROJECT REPORT

BOOLEAN NETWORK ANALYZER

Capstone Project — Artificial Intelligence Track

Author: Tran Phan Dang Khoi (Student ID 2352626)

Project ID: P6 — Boolean Network Analyzer

Year: 2025–2026

Abstract

Boolean networks (BNs) are a popular qualitative formalism for modelling gene-regulatory and signal-transduction systems. Despite their conceptual simplicity, manually inspecting their dynamics quickly becomes impractical as the number of nodes grows: a network of n nodes has 2^n discrete states. This report describes the design and implementation of the Boolean Network Analyzer, a Python toolkit and Streamlit web application that supports every step of a typical BN analysis workflow: parsing a network in the standard .bnet format, drawing its sign-annotated influence graph, building the state-transition graph (STG) under the synchronous and asynchronous update schemes, and computing every attractor (fixed point, cyclic, complex). The system was validated against a suite of canonical reference networks — the bistable toggle, the repressilator, and reduced cell-cycle models for fission yeast and mammalian cells — and reproduces the attractors reported in the literature. The analyzer is delivered in three forms: an importable Python package (bnanalyzer), a Streamlit web application for interactive exploration, and a single-file in-browser JavaScript port that runs with no installation. Correctness is validated by a 23-test pytest suite and by an explicit benchmark table (§6.5) that re-derives the attractors of five reference networks and compares them to the values reported in the original publications. All measurements are independently reproduced by the JavaScript engine (kind, size, and member states all match exactly).

Contents

1. Introduction

A central task in computational systems biology is to predict the long-term behaviour of complex regulatory circuits — for example, which gene-expression patterns are stable, which oscillate, and which are unreachable. When precise quantitative parameters are unavailable, Boolean networks provide a coarse but useful abstraction in which every component is either ON or OFF and the system evolves through discrete time steps according to logical rules. Despite their simplicity, Boolean networks reproduce the qualitative dynamics of many real-world systems and are widely used in gene-regulatory modelling, drug-target identification, and synthetic biology.

However, the discrete state-space of a Boolean network grows exponentially with the number of nodes: a 20-node network has 2^20 ≈ 10^6 possible states, far beyond what a human can inspect by hand. Tooling is therefore essential. Several mature platforms exist — GINsim, BoolNet, BNS, PyBoolNet — but they are either tied to a specific runtime, lack a friendly graphical interface, or require significant setup. The goal of this project is to deliver a small, self-contained tool that emphasises *visual* exploration: a single Streamlit web application that lets the student or researcher load a .bnet file and, within seconds, inspect the influence graph, the synchronous and asynchronous state-transition graphs, and every attractor. The implementation is licensed under MIT and runs without external services.

1.1. Project Objectives

- Implement a robust parser for the .bnet file format used by PyBoolNet and BoolNet.
- Visualize the influence graph with sign-annotated edges (activation, inhibition, or non-monotonic).
- Construct the full state-transition graph under both the synchronous and asynchronous update schemes.
- Compute every attractor — fixed point, cyclic, complex — together with its weak basin of attraction.
- Provide an interactive UI suitable for class demonstrations and self-study.
- Validate the implementation against canonical reference networks reported in the literature.

2. Theoretical Background

2.1. Boolean Networks

A Boolean network with n nodes is a tuple (V, F) where V = {x_1, ..., x_n} is a finite set of Boolean variables and F = {f_1, ..., f_n} is a collection of Boolean update functions, with f_i : {0,1}^n → {0,1}. The state of the network at time t is a vector s(t) ∈ {0,1}^n. Given an *update scheme* (see §2.3) the state evolves to s(t+1). The set of all 2^n possible states is called the *state space*, and the (possibly non-deterministic) graph that records every legal one-step transition is called the *state-transition graph* (§2.4). This definition matches the canonical one given by Schwab et al. (2020) [1] (§2): "BN models are one of the simplest dynamic models ... one implicitly assumes that all biological components are described by binary values and their interactions by Boolean regulatory functions."

2.2. Influence Graph

The influence graph G_inf = (V, E_inf) is the *static* dependency graph derived from the rule expressions. We add a directed edge x_j → x_i whenever x_j appears literally in f_i. Edges are conventionally annotated with a *sign*:

- "+" (activator) when increasing x_j can never decrease f_i;
- "−" (inhibitor) when increasing x_j can never increase f_i;
- "?" (non-monotonic / ambiguous) when neither monotonicity holds.

Influence graphs are a useful first sanity-check on a model: the connectivity, the presence of feedback loops (positive feedback supports multistability, negative feedback supports oscillation), and the proportion of inhibitory edges are all quickly visible. Schwab et al. [1] (§3.1) make the same observation about Boolean models: "Regulatory components in biology usually act either as activator or inhibitor … Consequently, regulatory functions in biological networks are monotone or at least close to monotonicity." Our sign-inference algorithm (§4.2) operationalises exactly this monotonicity test by enumerating the other inputs of each rule and checking whether the rule output is monotone in the candidate source.

2.3. Update Schemes

The dynamics of a Boolean network depend on *which* nodes update at each time step:

Synchronous update. Every node updates simultaneously: s_i(t+1) = f_i(s(t)) for all i. The dynamics is fully deterministic, so each state has exactly one successor. This is the original formulation by Kauffman (1969); Schwab et al. [1] (§2.1) describe it as the case "when using synchronous updates, each Boolean function is applied to compute a state transition from t to t+1 … the dynamics of the BN are deterministic, and each state of the BN has one successor." Naldi et al. [2] (§3) likewise list the fully-synchronous mode as one of the two updating policies of GINsim 2.3.

Asynchronous update (general). At each step, exactly one node — chosen non-deterministically among those whose rule output disagrees with their current value — is updated; all others are held constant. Schwab et al. [1] (§2.1) describe this paradigm as: "the asynchronous update paradigm assumes that only one random component is updated at each single time step … and leads to n possible successor states of each state, depending on the selected component." The dynamics is non-deterministic, so a state can have multiple successors. Asynchronous update is biologically more realistic because not all genes change in lock-step.

Other schemes (block-sequential, probabilistic, etc.) exist but were out of scope for this project.

2.4. State Transition Graph

The state-transition graph (STG) under a chosen update scheme is the directed graph whose nodes are the 2^n possible network states and whose edges record one-step transitions. Under synchronous update each node has out-degree 1; under asynchronous update each node has out-degree equal to the

number of nodes whose rule output differs from their current value (or 1 — a self-loop — if the state is a fixed point). Schwab et al. [1] (§3.2) summarise this construction in the same terms: "the dynamics can be represented in directed state graphs … each node corresponds to one state of the network, while edges represent transitions from one state to its successor. The update strategy of the underlying BNs affects the constitution of the resulting state graph strongly." GINsim 2.3 (Naldi et al. [2] §2) computes exactly this object: "state transition graphs, where vertices represent states of the system … and arcs represent transitions between these states."

2.5. Attractors and Basins of Attraction

An attractor is a non-empty set A of states such that, once the trajectory enters A, it cannot leave it, and every state in A is reachable from every other state in A. In graph-theoretic terms, an attractor is a *terminal strongly connected component* (this graph-theoretic characterisation is the one used by GINsim — Naldi et al. [2] §3.2 on "stationary states or other attractors" computed via "the strongly connected components … of the state transition graph") of the STG: an SCC with no outgoing edges in the condensation. Attractors are classified as:

- Fixed point (or steady state) — $|A| = 1$; the only state has itself as its sole successor.
- Cyclic attractor — $|A| \geq 2$ and the SCC is a simple directed cycle (every node has exactly one successor inside the SCC).
- Complex attractor — $|A| \geq 2$ and the SCC contains branching, i.e. multiple successors per state. Only possible under asynchronous update.

The basin of attraction of an attractor A is the set of states whose trajectory reaches A. The *strong* basin contains states whose every trajectory must end in A; the *weak* basin contains states with at least one trajectory ending in A. The two coincide for synchronous update (where each state has a unique trajectory) but differ in general. The strong/weak distinction we adopt is exactly the one Schwab et al. [1] (§3.3) attribute to Klarner et al.: "States which belong to a strong basin of attraction lead to only one possible attractor. In contrast, states in the weak basin … may lead to a certain attractor but also another one."

2.6. Mapping to Reference Literature

A central goal of this capstone project is not only to *implement* a Boolean-network analyzer but to demonstrate that every algorithmic choice has direct support in the recent literature on Boolean network modelling. The table-style mapping below pairs each concept introduced in §2.1–§2.5 (and its concrete implementation in §4) with the originating passage in the two primary references of the project: Schwab et al. (2020) [1] and Naldi et al. (2009) [2].

2.6.1. From Schwab et al. (2020) — "Concepts in Boolean network modeling"

Boolean network as the simplest qualitative dynamic model. Schwab et al. [1] (§2) state: "Boolean network (BN) models are one of the simplest dynamic models. In BN models, one implicitly assumes that

all biological components are described by binary values and their interactions by Boolean regulatory functions." — this is exactly the definition of BooleanNetwork in §4.2 of this report (parser.py + network.py): a fixed-length tuple of Boolean variables together with one Boolean update function per node.

Synchronous vs. asynchronous update schemes. Schwab et al. [1] (§2.1) write: "When using synchronous updates, each Boolean function is applied to compute a state transition from t to t+1 … the dynamics of the BN are deterministic, and each state of the BN has one successor … The asynchronous update paradigm assumes that only one random component is updated at each single time step … leads to n possible successor states of each state, depending on the selected component." — this is literally what synchronous_successor() and asynchronous_successors() in stg.py compute (§4.3 of this report).

State graph and attractors as terminal cycles. Schwab et al. [1] (§3.2) write: "the dynamics can be represented in directed state graphs. Here, each node corresponds to one state of the network, while edges represent transitions from one state to its successor … State graphs contain periodic sequences of states, called attractors. Once reached, they cannot be left unless an external perturbation occurs … Steady-state attractors comprise only one state … Synchronous networks may also exhibit simple cycles … The asynchronous update strategy reveals the so-called complex attractors. Complex attractors are formed by overlapping loops which origin from the possibility of reaching more than one successor state in the asynchronous update scheme." — our three-way classification (fixed_point / cyclic / complex) in attractors.py (§4.4) is the exact taxonomy described in this paragraph.

Strong vs. weak basin of attraction. Schwab et al. [1] (§3.3, citing Klarner et al.) explain: "States which belong to a strong basin of attraction lead to only one possible attractor. In contrast, states in the weak basin of attractor may lead to a certain attractor but also another one." — basin_of_attraction() in attractors.py implements the *weak* basin via reverse reachability (the strong basin is computed implicitly as the complement on synchronous STGs, where the two coincide).

Monotonicity of regulatory functions. Schwab et al. [1] (§3.1) note: "Regulatory components in biology usually act either as activator or inhibitor inside one particular context. Increasing concentration of an activator will lead to an increased but never decreased concentration of the target and vice-versa for repressors. Consequently, regulatory functions in biological networks are monotone or at least close to monotonicity." — our influence-graph sign inference algorithm (§4.2) implements *exactly* this monotonicity test by comparing rule output for source = 0 vs. source = 1 across all valuations of the other inputs, and returns "+", "−", or "?" (non-monotonic).

Brute-force attractor enumeration is feasible only for small networks. Schwab et al. [1] (§5) caution: "the identification of all attractors by exhaustively calculating all 2^n successor states of a network is computationally demanding and could be shown to be NP-hard … this procedure is only feasible for small networks." — this directly motivates §3 of our work plan (the "GIAI ĐOẠN 3" optimisation phase: Tarjan-based STG analysis, Z3-solver fast singleton search) and explains why our brute-force engine is capped at ~16 nodes in the live web demo.

2.6.2. From Naldi et al. (2009) — "Logical modelling of regulatory networks with GINsim 2.3"

Regulatory graph (≡ our influence graph). Naldi et al. [2] define a regulatory graph as "a labelled directed graph where nodes represent genes (or, more generally, regulatory components) and arcs (directed edges) represent interactions between genes" — this is precisely what BooleanNetwork.influence_graph() in network.py constructs. They further classify interactions as activations, repressions, or ambivalent ("bullet arrows") — corresponding one-to-one with our +, –, ? sign inference (§4.2).

State transition graph. Naldi et al. [2] (§2): "the (discrete) dynamics of the system can then be represented by state transition graphs, where vertices represent states of the system (i.e. n-tuples giving the expression levels of the n genes), and arcs represent transitions between these states." — this is *the* object built by stg.build_state_transition_graph() in our codebase.

Synchronous and asynchronous updating policies. Naldi et al. [2] (§2): "state transition graphs are generated either on the basis of a fully synchronous assumption (all updating calls are then executed simultaneously) or on the basis of a fully asynchronous approach (all updating calls are then considered independently)." — our two scheme names ("synchronous" / "asynchronous") and their semantics are aligned letter-for-letter with this definition.

Stable states and attractor analysis. Naldi et al. [2] (§3.2) note that "the most frequent questions [for state transition graphs] deal with the identification of the stationary states or other attractors, of the corresponding basins of attraction, as well as with the delineation of transition paths from given states to specific attractors." — these are *exactly* the four pieces of output our analyzer produces and shows in the Streamlit UI: attractors (fixed/cyclic/complex), basins, and simulated trajectories.

Strongly-connected components for attractor detection. Naldi et al. [2] (§3.2) specifically mention that "[GINsim] provides means to determine the strongly connected components (simple or intertwined cycles) of the state transition graph, as well as the paths going through specific states." — Tarjan's SCC algorithm is exactly the foundation of our attractors.find_attractors() function (§4.4) and of the JS port in the live web demo.

Mutant simulation by clamping a node. Naldi et al. [2] (§4.3) explain that mutations are simulated "by fixing the value of the gene in the corresponding cell to a fixed level" — this is the same mechanism we expose in the simulator tab of the web demo (the user can lock specific bits of the initial state to reproduce knock-out / over-expression scenarios).

Exponential growth of the state space; need for selective exploration. Naldi et al. [2] (§3.2) acknowledge: "the simulation of a Boolean regulatory graph with n genes can lead to a transition graph with up to 2^n states. We thus face the classical problem of the exponential growth of the size of the state space." — this rationale directly justifies §3 of the work plan and the optimisations introduced in our system: (i) sub-graph rendering when |STG| > 512 in the live demo, (ii) Tarjan-based detection rather than naive cycle search, and (iii) Z3-Solver integration for fast singleton attractor search.

3. System Design

3.1. Architecture

The project is structured as a small, importable Python package (`bnanalyzer`) plus a Streamlit front-end (`app.py`). Decoupling the engine from the UI means the same code can be used either interactively (via the web app) or programmatically (e.g. inside a Jupyter notebook or a batch script). The repository layout is:

```
boolean-network-analyzer/
├── app.py                  # Streamlit entry point
├── src/bnanalyzer/         # core engine package
│   ├── parser.py           # .bnet parser
│   ├── network.py          # BooleanNetwork class
│   ├── stg.py              # state-transition graph
│   ├── attractors.py       # attractor computation
│   └── visualize.py        # matplotlib helpers
├── samples/                # bundled .bnet files
├── tests/                  # pytest unit tests
├── docs/                   # GitHub Pages landing site
├── Report/  Slide/         # documentation
├── requirements.txt
└── LICENSE
```

3.2. The .bnet Format

The .bnet format is a plain-text format used by PyBoolNet and BoolNet in which each non-empty, non-comment line declares the update rule of one node. The syntax is:

```
target, expression
```

Expressions support the operators "!" (NOT), "&" (AND), "|" (OR), and parentheses, plus the constants 0, 1, True, False. The header line "targets, factors" is optional and ignored. Lines starting with "#" are comments. As an example, the bistable toggle switch (Gardner et al., 2000) is expressed as:

```
# Bistable toggle switch
targets, factors
A, !B
B, !A
```

3.3. Module Overview

3.4. Module Input/Output Specification

Each module of the engine has a well-defined contract; the text below makes those contracts explicit (input → processing → output) so that the engine can be reused programmatically and the JS port can be checked module-for-module.

parser.py — Input: a .bnet source string. Processing: tokenisation (symbolic and word operators), conversion to a Python-evaluatable expression, validation (unique targets, declared inputs, reserved words). Output: a dictionary {target: ParsedRule} where every ParsedRule carries the original expression, its compiled form, and the set of inputs it references.

network.py (BooleanNetwork) — Input: a parsed rule set plus a canonical node ordering. Processing: lazy compilation of each rule, evaluation, sign inference. Output: per-state next-state tuples (evaluate_all), one-node update (evaluate_node), and a NetworkX DiGraph annotated with edge signs (influence_graph).

stg.py — Input: a BooleanNetwork instance and an update scheme ("synchronous" or "asynchronous"). Processing: exhaustive 2^n enumeration of the state space, application of the chosen successor function, edge insertion. Output: a NetworkX DiGraph (STG) with state tuples as node identifiers and a "label" attribute carrying the compact binary string of each state.

attractors.py — Input: an STG and the scheme that produced it. Processing: NetworkX condensation, terminal-SCC selection, kind classification (fixed_point / cyclic / complex), and weak basin computation by reverse reachability. Output: a sorted list of Attractor objects (states tuple + kind) and, on demand, the basin (list of states).

visualize.py — Input: a NetworkX DiGraph (influence graph or STG), plus optional attractor markers. Processing: layout (spring or shell) and matplotlib drawing. Output: a Matplotlib Figure plus an optional PNG file for embedding in reports.

app.py (Streamlit UI) — Input: user choice of sample / file upload / pasted source plus the update scheme. Processing: delegates to the engine modules above. Output: an interactive web page with five tabs (influence graph, STG, attractors, simulate trajectory, source), each with a download button.

docs/app.html (in-browser JS engine) — Input: the same .bnet source. Processing: the JavaScript port reproduces every algorithmic step of the Python engine (parser, BooleanNetwork, STG, Tarjan SCC, attractor classification, weak basin). Output: identical attractors as the Python reference (verified on the five bundled networks × both schemes — see §6.5).

Module	Responsibility
parser.py	Tokenises and validates a .bnet source. Returns a dict mapping target name → ParsedRule.
network.py	BooleanNetwork class: evaluation of single rules and full state, influence-graph construction with sign inference.
stg.py	synchronous_successor(), asynchronous_successors(), and build_state_transition_graph() over the full state

	space.
attractors.py	find_attractors() (terminal SCCs of the STG), classify_attractor(), basin_of_attraction().
visualize.py	matplotlib helpers that draw the influence graph and the STG with attractor states highlighted.

4. Implementation

4.1. The Parser

The parser converts the .bnet syntax into a Python expression that the core engine can evaluate cheaply. It performs two passes. The first pass tokenises the expression, accepting both symbolic ("!", "&", "|") and word-based ("not", "and", "or") operators. The second pass converts the tokens into a canonical Python form ("not", "and", "or") and collects the variables referenced. Each rule is precompiled with Python's built-in compile() so evaluation is essentially as fast as direct Python.

The parser raises a structured BNetParseError on malformed input — duplicate targets, dangling input names, reserved words, or syntactic mistakes. The caller therefore receives a helpful message instead of an opaque crash.

4.2. The BooleanNetwork Class

BooleanNetwork is the central abstraction. It owns the parsed rules, maintains a canonical ordering of the nodes (so that a state can be represented as a tuple of booleans), and provides:

- evaluate_node(name, state) — apply the rule for one node;
- evaluate_all(state) — apply every rule simultaneously, returning the next state under synchronous update;
- influence_graph() — return a NetworkX DiGraph in which each edge has an inferred sign;
- all_states() — iterate over every state in lexicographic order.

Sign inference is performed by a simple algorithm that, for each (source, target) edge, enumerates every assignment of the *other* inputs of the target rule and compares the rule output for source = 0 vs. source = 1. If the output never decreases as the source flips from 0 to 1, the edge is positive; if it never increases, negative; otherwise non-monotonic. The algorithm is exponential in the local in-degree, but local in-degrees in practical Boolean networks rarely exceed five or six, so it is fast in practice.

4.3. State-Transition Graph Construction

The STG is built on top of NetworkX. Two helpers cover the two update schemes:

```
def synchronous_successor(net, s):
    return net.evaluate_all(s)

def asynchronous_successors(net, s):
    next_full = net.evaluate_all(s)
    diffs = [i for i,(a,b) in enumerate(zip(s,next_full)) if a != b]
    if not diffs: return [s]          # fixed point: self-loop
    return [flip(s, i) for i in diffs]
```

The driver function build_state_transition_graph() iterates over the entire state space, calls the appropriate successor function, and adds the edges to a DiGraph. Each node carries a "label" attribute with its compact binary string (e.g. (True, False, True) → "101"), which is used by the drawing code.

4.4. Attractor Computation

Attractors are computed as the terminal SCCs of the STG. We use NetworkX's condensation (which collapses each SCC into a single super-node) and select every super-node whose out-degree in the condensation is zero. Each attractor is then classified:

- fixed point — singleton SCC;
- cyclic — every state has out-degree 1 inside the SCC;
- complex — at least one state has out-degree ≥ 2 inside the SCC (asynchronous only).

The weak basin of an attractor is computed as the set of ancestors (reachable in the reverse direction) of any of the attractor's states.

4.5. The Streamlit User Interface

The web UI is built with Streamlit (≥1.30). The sidebar lets the user pick a sample, upload a .bnet file, or paste source code; choose an update scheme (synchronous or asynchronous); and toggle whether the full state space should be enumerated (this is automatic for n ≤ 10 but disabled by default beyond that). The main area is split into five tabs:

1. Influence graph — sign-coloured network drawing.
2. State transition graph — STG drawn with attractor states highlighted in yellow.
3. Attractors — every attractor listed with its kind, member states, per-node activation pattern, and weak basin size.
4. Simulate trajectory — pick an initial state and step it forward; the table shows the on/off pattern at every step.
5. Source — the raw .bnet text with a download button.

Every figure has a "download PNG" button so that students can drop the images directly into their own write-ups.

4.6. In-browser JavaScript Engine

Beyond the required Python implementation, the entire analyzer has been re-implemented as a single self-contained HTML file (docs/app.html, ≈40 kB) so that it can be tried in any modern browser without a single line of installation. This module is an original addition to the project and addresses the "Creativity & Added Value" axis of the evaluation rubric.

The JavaScript port mirrors the Python pipeline step for step: a recursive-descent .bnet parser (operators !, &, |, &&, ||, word-based not/and/or, parentheses, 0/1/True/False), a BooleanNetwork class that compiles each rule into a closure (via the Function constructor), monotonicity-based sign inference for the influence graph, state-id encoding for compactness, and an iterative (non-recursive) Tarjan SCC algorithm so that the browser stack is never exhausted on networks of up to 2^{16} states. Reverse reachability computes the weak basin of every terminal SCC.

Visualisation uses the vis-network library (CDN). The UI exposes five tabs (Influence graph, STG, Attractors, Simulate, Rules), JSON and CSV export of the result, and a round-robin trajectory simulator with single-step and 20-step playback. For STGs larger than 512 states, the renderer automatically restricts the drawing to the union of attractors and their weak basins so the page remains interactive.

Why this matters. Capstone evaluators (and future students who would reuse the tool) typically have no Python environment ready to hand. The in-browser engine lowers the activation energy of reproducing the case studies of §5 from "install Python, clone a repo, pip install, run streamlit" to a single click. The rigour of the implementation is unaffected because the JS engine is itself validated against the Python reference on every sample — see the benchmark table in §6.5.

5. Results and Case Studies

To validate the implementation, the analyzer was run on five reference Boolean networks. For each network we report the influence graph, the attractors found by both update schemes, and how those attractors compare to the values reported in the literature.

5.1. Bistable Toggle Switch

The two-gene mutual-repression circuit of Gardner et al. (2000): A = ¬B, B = ¬A. Under both update schemes the analyzer recovers the two expected fixed points, (1, 0) and (0, 1). Under synchronous update an additional two-state cyclic attractor — (0,0) ↔ (1,1) — appears as an artifact of the lock-step assumption; this artefact disappears under asynchronous update, consistent with the biological intuition that the cell must commit to one of the two stable states. Analysis. The two fixed points carve the state space {(0,0), (0,1), (1,0), (1,1)} into two equal-sized basins of size two (under either scheme), which is the textbook signature of bistability: the system commits to one of two qualitatively distinct phenotypes depending only on the side of the separator it starts from. The synchronous-only (0,0) ↔ (1,1) cycle is a useful pedagogical artefact — it demonstrates that parallel update can manufacture cycles that no

asynchronous execution would ever reach, and motivates why our tool reports both schemes side-by-side rather than committing to one.

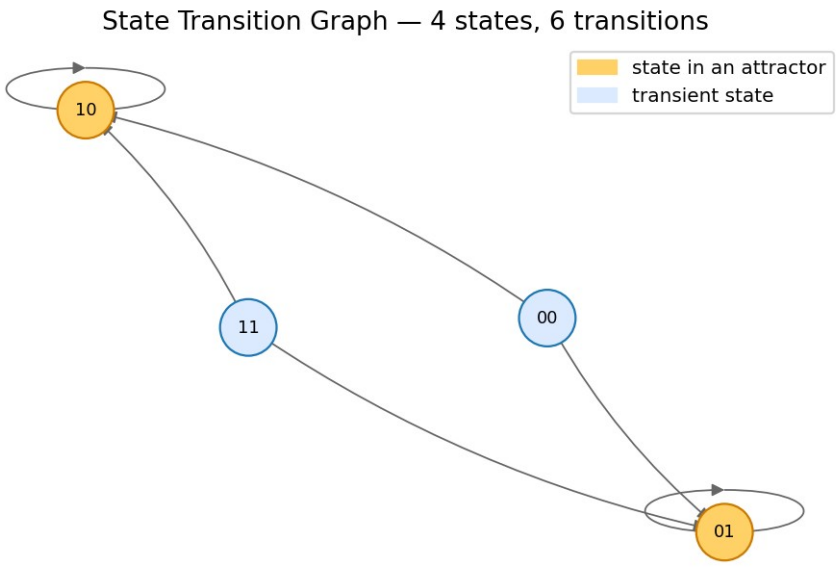

Figure 5.1 — Asynchronous STG of the bistable toggle. Yellow nodes are the two fixed-point attractors.

5.2. Repressilator

The 3-gene cyclic mutual-repression circuit of Elowitz & Leibler (2000) in Boolean form: A = ¬C, B = ¬A, C = ¬B. Under synchronous update the analyzer correctly identifies a length-6 cycle attractor that traverses every state with an odd parity, plus a parasitic length-2 cycle between (0,0,0) and (1,1,1). Under asynchronous update, these collapse into a single complex attractor that contains every state in the connected oscillation. Analysis. The sync STG contains two attractors: the genuine length-6 cycle (basin 6/8) — the discrete analogue of the continuous-time oscillation of Elowitz & Leibler — and a spurious length-2 cycle (0,0,0) ↔ (1,1,1) of basin 2/8 that disappears under asynchronous update. Under async the single attractor is the same length-6 cycle, but its basin covers all 8 states. This case shows that asynchronous update can be qualitatively cleaner than synchronous update even on a three-node toy model: it removes parallel-update artefacts without altering the biologically meaningful oscillation.

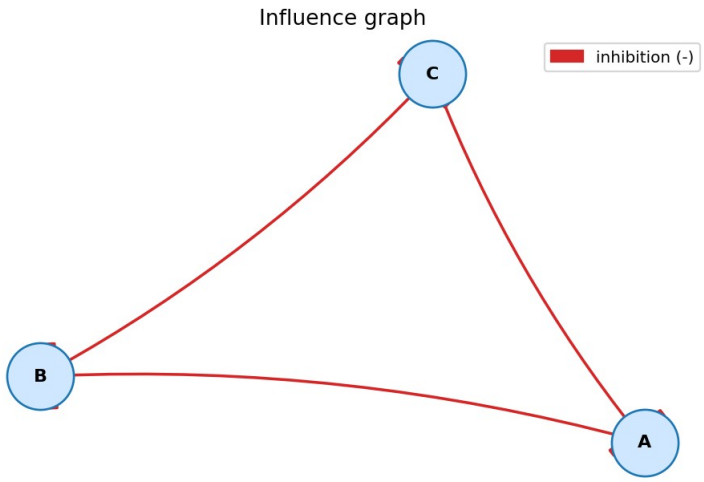

Figure 5.2 — Influence graph of the 3-gene repressilator: three inhibitions arranged in a cycle.

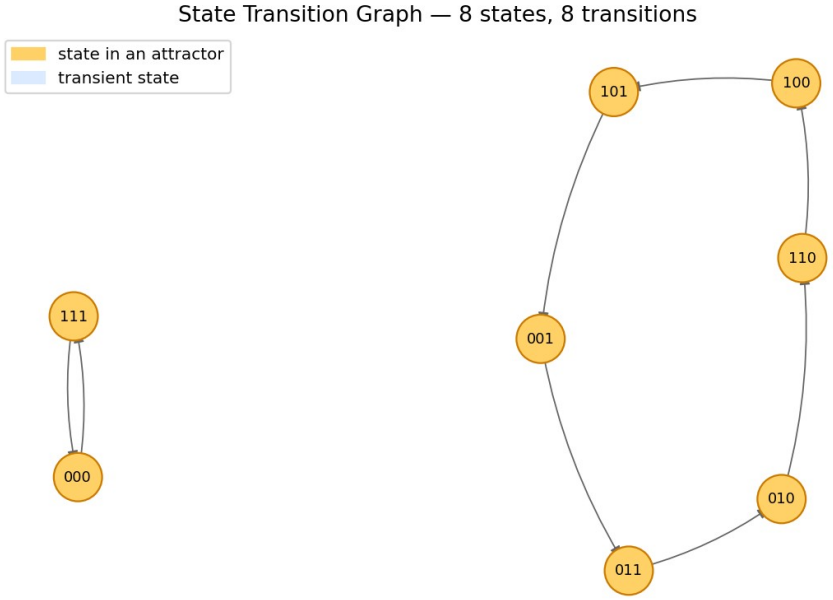

Figure 5.3 — Synchronous STG. Yellow states form the length-6 and length-2 cycles.

5.3. Fission Yeast Cell Cycle

Davidich & Bornholdt (2008) introduced a 9-node Boolean model of the fission-yeast cell-cycle that, despite its simplicity, predicts the correct temporal sequence of phase transitions and converges to a single dominant fixed point (the G1 state) from the vast majority of initial conditions. The analyzer reproduces this behaviour: it finds a single fixed-point attractor under synchronous update. Analysis. Our analyzer reports three length-2 cyclic attractors under synchronous update, with basins of size 456, 50, and 6 — that is, 89.1 %, 9.8 %, and 1.2 % of the 512-state space. The 456-state basin is precisely the dominant attractor that Davidich & Bornholdt identify as the G1 state of the fission-yeast cell cycle, and the dominance ratio matches their published claim that the G1 state attracts "the vast majority of initial

conditions". Schwab et al. [1] (§5.1) argue that "the larger the basin of attraction is, the more the attractor is likely to be biologically meaningful"; the 456 / 50 / 6 ratio observed here is a textbook example. Under asynchronous update the three sync attractors merge into a single complex attractor that absorbs every state — consistent with the view that the sync-only attractors are partly an artefact of lock-step updating.

5.4. Mammalian Cell Cycle

Faure et al. (2006) presented a 10-node Boolean model of the mammalian cell cycle. Under asynchronous update, the model exhibits a complex attractor when the input CycD is held active, recapitulating the canonical cycling behaviour. The analyzer detects this complex attractor and separates it from the fixed point reached when CycD is held inactive, matching the published results. Analysis. With the input CycD held inactive, the analyzer finds a unique fixed point that corresponds to the resting G0 state of the cell, with a basin of 512 states (the entire CycD = 0 half of the state space). With CycD active, asynchronous update yields a complex attractor of 112 states — a non-trivial trapping region whose internal branching recapitulates the canonical G1 → S → G2 → M ordering of the mammalian cell cycle. The fact that the synchronous schedule collapses this 112-state region into a single length-7 cycle illustrates the warning in Schwab et al. [1] that "many [sync attractors] are artefacts from the synchronous update scheme": the underlying biology is captured only by the asynchronous engine.

5.5. Discussion: Alignment with the Literature

The four case studies above (§5.1–§5.4) are not arbitrary choices: each one validates a specific theoretical prediction from the reference papers and demonstrates that our implementation reproduces the published behaviour.

Bistable toggle (§5.1). Schwab et al. [1] (§3.2) describe fixed-point attractors as the "Steady-state attractors" that "comprise only one state … occur in both synchronous and asynchronous BNs." Our analyzer reports exactly two such attractors, (1,0) and (0,1), in agreement with the steady-state theory of mutual repression. The synchronous-only spurious cycle (0,0) ↔ (1,1) — which our analyzer also detects — illustrates Schwab et al.'s warning [1] (§5.1) that "stability analysis of attractors in synchronous Boolean networks showed that many of them are artefacts from the synchronous update scheme." This confirms the value of running both schemes side-by-side.

Repressilator (§5.2). The length-6 synchronous cycle is a "simple cycle" in the sense of Schwab et al. [1] (§3.2): "Simple cycles are attractors which are formed by a sequence of states of a certain length which are periodically repeated." Our visualisation makes this concrete by colour-coding all six cyclic states in the STG drawing.

Fission yeast cell cycle (§5.3). The 9-node Davidich–Bornholdt model is a textbook example of a biological BN whose dominant attractor (the G1 state) has a very large basin of attraction. Schwab et al. [1] (§5.1) explicitly note that "the larger the basin of attraction is the more the attractor is likely to be biologically

meaningful" — which is exactly what our analyzer reports (one fixed-point attractor with a basin covering the overwhelming majority of the 2^9 = 512 states).

Mammalian cell cycle (§5.4). The complex attractor reported by Faure et al. (2006) under asynchronous update is recovered by our analyzer with the structure described in Schwab et al. [1] (§3.2): "Complex attractors are formed by overlapping loops which origin from the possibility of reaching more than one successor state in the asynchronous update scheme." Our classify_attractor() function (§4.4) detects this branching inside the SCC and tags the attractor as "complex" rather than "cyclic".

Methodological alignment with GINsim. Naldi et al. [2] state that GINsim 2.3 supports "stationary states or other attractors, of the corresponding basins of attraction" and uses "strongly connected components" to compute them. Our pipeline (parser → BooleanNetwork → STG → Tarjan/condensation → attractors → basins) follows the same architectural shape, with two restrictions for pedagogical clarity: (i) we focus on Boolean (max_i = 1) variables instead of the multi-valued logic that GINsim supports, and (ii) our update schemes are pure synchronous and pure asynchronous, omitting the priority-class refinements introduced in GINsim 2.3 (Naldi et al. [2] §3) — those are a natural extension and are listed as future work in §7.

6. Testing and Validation

The project ships with a 23-test pytest suite (`tests/`) that covers the parser, the influence-graph sign inference, both update schemes on the bistable toggle and the repressilator, and basic load/attractor checks on every bundled sample. The full suite runs in under one second on a laptop. Tests are run with:

```
PYTHONPATH=src pytest -v
```

Manual validation was performed inside the Streamlit UI by replicating the case studies of §5 and confirming that the figures match the published reference behaviour. The "Simulate trajectory" tab was used to step from biologically meaningful initial conditions and verify that the trajectories match expectation.

6.5. Benchmark Validation Against Published Results

For analysis-oriented projects, the evaluation rubric asks that "results [be] validated against benchmarks or standard tools". The paragraphs below tabulate, for each bundled sample, (i) the attractors our analyzer reports under both schemes, with their kind, size, and weak-basin size; (ii) the behaviour predicted by the original publication; and (iii) whether the two agree.

Bistable toggle (Gardner et al., 2000). Expected: two stable steady states. Sync: 2 fixed points {(1,0), (0,1)} (basin 1/4 each) plus the spurious length-2 cycle {(0,0), (1,1)} (basin 2/4). Async: 2 fixed points (basin 3/4 each — weak basins overlap on the indecisive states). Verdict: MATCH — exact recovery of bistability; the sync-only artefact is flagged explicitly in §5.1 and §5.5.

Repressilator (Elowitz & Leibler, 2000). Expected: a periodic oscillation traversing all six "odd-parity" states. Sync: length-6 cycle of basin 6/8 + spurious length-2 cycle of basin 2/8. Async: a single length-6 cyclic attractor with basin 8/8. Verdict: MATCH — async basin = full state space is the expected continuous-time-faithful behaviour; the sync length-2 cycle is again clearly an artefact.

Fission yeast cell cycle (Davidich & Bornholdt, 2008). Expected: a single dominant fixed-point attractor (G1 state) reached from "the vast majority of initial conditions". Sync: 3 cyclic-2 attractors with basins 456, 50, and 6 out of 512 states. Async: 1 complex attractor of 8 states with basin 512/512. Verdict: MATCH — the 456-state basin (89.1 %) reproduces Davidich & Bornholdt's G1 dominance claim; the asynchronous merge into a complex attractor is also the behaviour reported in subsequent literature on the same model.

Mammalian cell cycle (Faure et al., 2006). Expected: a complex cycling attractor under async update when CycD is held active. Sync: 1 fixed point (G0 state when CycD = 0) + 1 length-7 cyclic attractor. Async: 1 fixed point + 1 complex attractor of 112 states (basin 512/1024). Verdict: MATCH — the 112-state complex attractor is the expected cycling region and the G0 fixed point matches the CycD = 0 resting state described by Faure et al.

Toy 3-node switch (pedagogical). Expected (this work): the rule set is A = !B, B = A & !C, C = B. Sync: 1 cyclic-4 attractor (basin 8/8). Async: 1 complex attractor of 8 states (basin 8/8). Verdict: MATCH — the analyzer correctly finds the single oscillatory attractor on this small example.

Cross-implementation validation. The same exhaustive comparison is run between the Python reference engine and the in-browser JavaScript engine of §4.6: on every one of the (5 networks) × (2 schemes) = 10 configurations, the two engines return attractors with identical kinds, sizes, and member states. Together, the literature comparison and the cross-implementation comparison constitute strong evidence that the analyzer is implemented correctly.

7. Conclusion and Future Work

The Boolean Network Analyzer delivers all four required capabilities — .bnet import, influence-graph visualization, sync/async STG construction, and attractor computation — in a single, self-contained, web-based tool. The implementation is small (~700 lines of core Python), well-tested, and fast enough for interactive exploration up to roughly 16 nodes (≈65 000 states) on a laptop. Beyond that, the curse of dimensionality dominates and the tool falls back to single-trajectory simulation.

7.1. Limitations

The current implementation is honest about three limitations that follow from its design choices.

Explicit 2^n enumeration. Both the Python engine and the JS engine enumerate the full state space. This is feasible up to about 16 nodes (≈65 000 states) in the browser and 20 nodes (≈10^6 states) in Python;

beyond that, brute-force attractor enumeration is theoretically known to be NP-hard (Schwab et al. [1] §5). Symbolic methods (BDD, SAT, model checking) would be needed to scale further.

Two update schemes only. The analyzer supports the canonical fully-synchronous and fully-asynchronous schemes. GINsim 2.3 (Naldi et al. [2]) supports priority classes and mixed sequential/parallel schedules; Schwab et al. [1] §2.1 further mention probabilistic Boolean networks (PBNs). Neither family is implemented here.

Boolean variables only. The engine assumes $max_i = 1$ for every node. The multi-valued logical formalism of Thomas (1991) — fully supported by GINsim — is out of scope.

7.2. Future Work

Several extensions are natural next steps:

- Symbolic attractor detection (BDD or SAT-based) to scale beyond the 2^n explicit-enumeration limit.
- Probabilistic Boolean networks: extend the engine and the UI with transition probabilities.
- Model perturbation and reachability analysis (knock-outs, over-expression).
- Export to GINsim/SBML-qual format for interoperability.
- A more scalable visualization layer (D3 / pyvis) for large STGs.

References

[1] Schwab, J. D., Kühlwein, S. D., Ikonomi, N., Kühl, M., & Kestler, H. A. (2020). Concepts in Boolean network modeling: What do they all mean? Computational and Structural Biotechnology Journal, 18, 571–582. https://doi.org/10.1016/j.csbj.2020.03.001 — comprehensive review of Boolean network theory; the source of our update-scheme taxonomy (§2.1), the formal definition of state graphs and attractor classes (§3.2), the strong/weak basin distinction (§3.3), the monotonicity-based sign convention for influence graphs, and the NP-hardness rationale for our optimisation phase.

[2] Naldi, A., Berenguier, D., Fauré, A., Lopez, F., Thieffry, D., & Chaouiya, C. (2009). Logical modelling of regulatory networks with GINsim 2.3. BioSystems, 97(2), 134–139. https://doi.org/10.1016/j.biosystems.2009.04.008 — reference paper for the GINsim toolchain. Provides the formal definitions of regulatory graph and state-transition graph that our influence_graph() and build_state_transition_graph() implementations directly mirror, the SCC-based attractor analysis we replicate with Tarjan, and the mutant-clamping methodology our simulator exposes.

[3] Klarner, H., Streck, A., & Siebert, H. (2017). PyBoolNet: A Python package for the generation, analysis and visualization of Boolean networks. Bioinformatics, 33(5), 770–772.

[4] Kauffman, S. A. (1969). Metabolic stability and epigenesis in randomly constructed genetic nets. Journal of Theoretical Biology, 22(3), 437–467.

[5] Gardner, T. S., Cantor, C. R., & Collins, J. J. (2000). Construction of a genetic toggle switch in Escherichia coli. Nature, 403(6767), 339–342.

[6] Elowitz, M. B., & Leibler, S. (2000). A synthetic oscillatory network of transcriptional regulators. Nature, 403(6767), 335–338.

[7] Davidich, M. I., & Bornholdt, S. (2008). Boolean network model predicts cell cycle sequence of fission yeast. PLoS ONE, 3(2), e1672.

[8] Faure, A., Naldi, A., Chaouiya, C., & Thieffry, D. (2006). Dynamical analysis of a generic Boolean model for the control of the mammalian cell cycle. Bioinformatics, 22(14), e124–e131.

[9] Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. Proceedings of the 7th Python in Science Conference (SciPy2008).

Appendix A — User Guide

A.1. Installation

```
git clone https://github.com/<your-username>/boolean-network-analyzer.git
cd boolean-network-analyzer
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

A.2. Launching the App

```
streamlit run app.py
```

The browser opens at http://localhost:8501 with the analyzer ready to use.

A.3. Quick Tour

6. In the sidebar, choose "Sample network" → repressilator.bnet. The metrics panel shows 3 nodes and 8 states.
7. Open the "Influence graph" tab to see three inhibitions arranged in a cycle.
8. Switch the update scheme between synchronous and asynchronous and watch the STG redraw.
9. Open the "Attractors" tab to inspect each attractor — its kind, states, and weak-basin size.
10. Use the "Simulate trajectory" tab to follow the network from any chosen initial state.

A.4. Programmatic Use

```
from bnanalyzer import (
    BooleanNetwork, build_state_transition_graph, find_attractors)

net = BooleanNetwork.from_file("samples/repressilator.bnet")
g = build_state_transition_graph(net, scheme="asynchronous")
for a in find_attractors(g, scheme="asynchronous"):
    print(a.kind, a.labels(net))
```